OpenMP — Lesson 1

The parallel region and the fork–join model

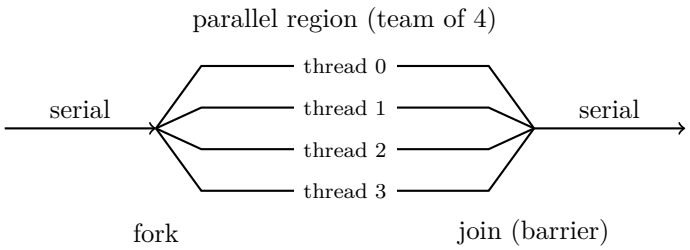
Study notes for Flor Vargas

November 2026

Sources: Chapman, Jost & van der Pas [1], Ch. 2 "Overview of OpenMP" (fork–join execution model) and Ch. 3 "Writing a First OpenMP Program"; the `parallel` construct in detail is in Ch. 4 "OpenMP Language Features".

1 The idea

An OpenMP program starts as an ordinary serial C program with **one** thread, the *master* (id 0). When it reaches a `parallel` directive it **forks**: a *team* of threads is created and *every* thread executes the same block of code. At the end of the block there is an **implicit barrier**: all threads wait until the whole team has finished, then the team **joins** back into the master, and the program continues serially.



2 The three ingredients

1. **The directive** `#pragma omp parallel` placed before a block `{ ...}`. This is the only construct that creates threads.
2. **The runtime library** `omp.h`: `omp_get_thread_num()` returns the calling thread's id $(0, \ldots, n-1)$; `omp_get_num_threads()` returns the team size n.
3. **The compiler flag** `-fopenmp`. Without it, `gcc` ignores the pragma.

The team size is chosen at run time, most simply with the environment variable `OMP_NUM_THREADS`. If it is not set, the default is usually the number of cores.

3 The code

`hello_parallel.c`

```c
/*
 * hello_parallel.c -- OpenMP Lesson 1: the parallel region (fork-join).
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 2 (fork-join execution model) and Ch. 3 (first OpenMP program).
```

```c
 *
 * Compile:   gcc -Wall -fopenmp -o hello_parallel hello_parallel.c
 * Run:       ./hello_parallel
 *            OMP_NUM_THREADS=4 ./hello_parallel
 */

#include <stdio.h>
#include <omp.h>    /* OpenMP runtime library: omp_get_thread_num(), ... */

int main(void)
{
    /* ---- SERIAL PART: only one thread exists (the "master", id 0) ---- */
    printf("before: %d thread(s) running\n", omp_get_num_threads());

    /* ---- FORK: a team of threads is created here ----
       Every thread executes the whole block { ... } below.            */
    #pragma omp parallel
    {
        /* Declared INSIDE the region -> each thread has its own copy. */
        int id = omp_get_thread_num();     /* 0, 1, ..., n-1           */
        int n  = omp_get_num_threads();    /* size of the team         */

        printf("  hello from thread %d of %d\n", id, n);
    }
    /* ---- JOIN: implicit barrier. The master waits here until every
       thread has finished the block; then the team is dissolved.     */

    /* ---- SERIAL PART again: back to one thread ---- */
    printf("after:  %d thread(s) running\n", omp_get_num_threads());

    return 0;
}
```

```
gcc -Wall -fopenmp -o hello_parallel hello_parallel.c
OMP_NUM_THREADS=4 ./hello_parallel
```

Typical output (your order will differ):

```
before: 1 thread(s) running
  hello from thread 1 of 4
  hello from thread 3 of 4
  hello from thread 2 of 4
  hello from thread 0 of 4
after:  1 thread(s) running
```

Key ideas

- Code *outside* the region runs once; code *inside* runs once *per thread*.
- The threads run concurrently, so the order of their output is **not deterministic**. Never assume thread 0 goes first.
- The barrier at the end of the region guarantees that "after" is printed only when every thread is done.
- Variables declared *inside* the region are private to each thread. That is why every thread prints its own `id`. (Data sharing is Lesson 2.)

4 The experiment

Experiment

Question: how does the team size control the work, and is the execution order reproducible?

1. Check how many cores your machine has: `nproc`.
2. Run the program with several team sizes:

```
for t in 1 2 4 8; do echo "== $t threads"; OMP_NUM_THREADS=$t ./hello_parallel; done
```

3. Repeat the 4-thread run five times:

```
for r in 1 2 3 4 5; do OMP_NUM_THREADS=4 ./hello_parallel; echo; done
```

What to observe and answer.

1. The number of "hello" lines equals `OMP_NUM_THREADS`, even when it is larger than `nproc`. What does that tell you about threads vs. cores?
2. "before" and "after" always print once and report 1 thread. Why?
3. Does the order of the hello lines change between the five repeated runs? Why can't OpenMP promise an order?

Answers (read after trying). (1) Threads are a software concept; the operating system time-shares them on the available cores. Asking for more threads than cores is allowed but usually gives no speed-up. (2) Those lines are outside the parallel region, where only the master exists. (3) The order usually changes. Each thread is scheduled independently by the OS, so the interleaving of their `printf` calls depends on timing. Any program whose *result* depends on that timing has a *race condition*, the central danger of shared-memory programming.

Next lesson. Shared vs. private variables: what happens when the threads write to the *same* variable (`shared`, `private` clauses; Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.